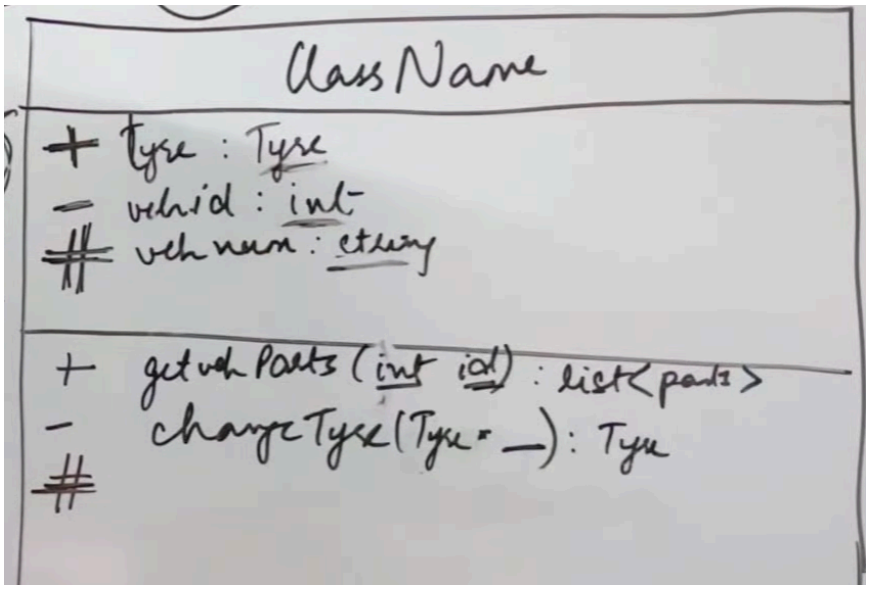
Basics of Low-Level Design (LLD) & UML Class Diagrams

Detailed notes based on Keerti Purswani's explanation of UML Class Diagrams, Association, Aggregation, and Composition.

1. The Importance of Class Diagrams

- **Why draw before coding?** Jumping straight into writing code without a design often leads to confusion, complex refactoring, and constant context-switching between files.
- **Clarity:** UML (Unified Modeling Language) diagrams help developers structure classes, determine storage requirements, establish connections, and implement SOLID principles clearly.
- **Interviews & Real-world usage:** In LLD interviews, interviewers generally demand to see an Object-Oriented Design (OOD) before any code is written to ensure you understand how classes interact, define data access levels, and map references.

2. Structure of a Class Box



In a UML class diagram, a class is represented by a large rectangular box divided into three horizontal sections:

- **Top Section (Class Name):** Contains the name of the class (e.g., Vehicle). If the entity is an interface, its name is typically written in *italics* or denoted with brackets like <<InterfaceName>>.
- **Middle Section (Attributes/Properties):** Contains the data variables or properties of the class. Format: attributeName : DataType (e.g., id : Integer, tire : Tire).
- **Bottom Section (Methods/Operations):** Contains the functions or behaviors of the class. Format: methodName(parameterName : DataType) : ReturnType (e.g.,

changeTire(id : Integer) : Void, getParts() : List).

Access Modifiers

Symbol	Modifier	Description
+	Public	Accessible from anywhere.
-	Private	Accessible only within the class.
#	Protected	Accessible within the class and its child/derived classes.

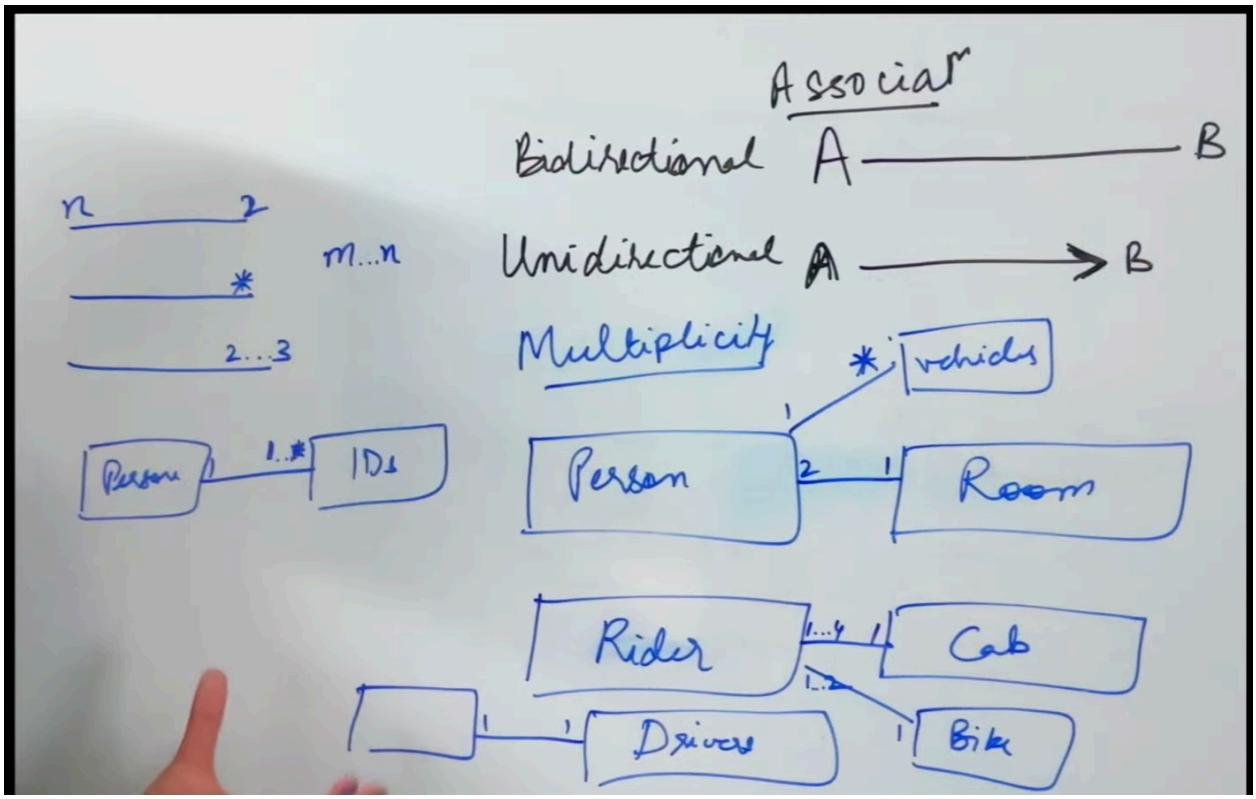
Static Members

If an attribute or a method is underlined, it means it is a **Static** member of the class.

3. Relationships Between Classes

Arrows and lines connecting the class boxes dictate how classes interact, references they hold, and how lifecycles are managed.

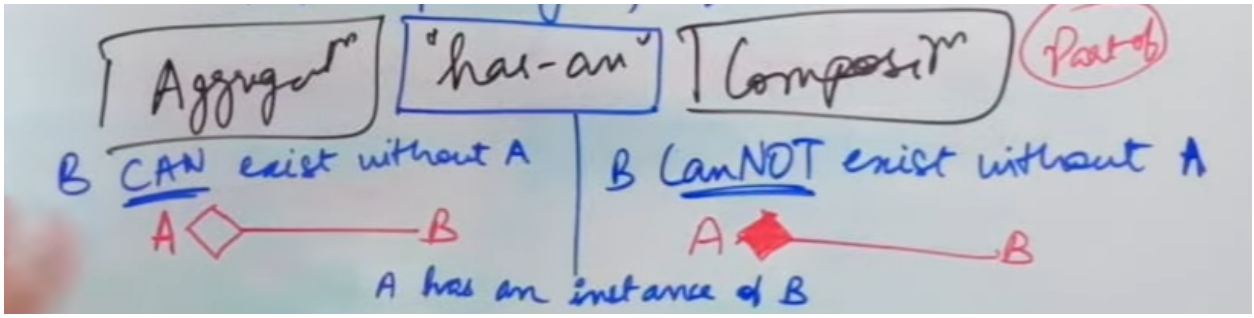
A. Association ("Uses" Relationship)

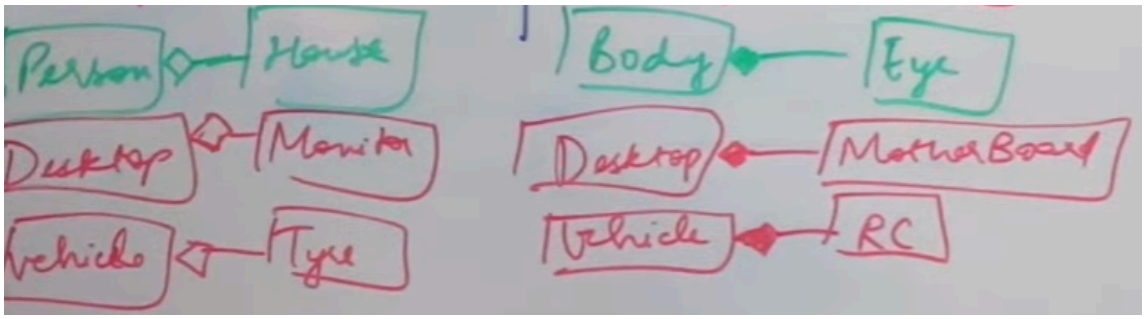


A basic relationship indicating that two classes interact with each other.

- **Bi-directional Association (Simple solid line):** Both classes can call and access each other. (e.g., Train and TrainStation).
- **Uni-directional Association (Solid line with an open arrow):** Navigation only happens in one direction. Class A can access Class B, but B cannot access A. (e.g., A Person can access a Room).
- **Multiplicity:** Denotes how many instances of a class relate to another. Values are written above the line at the ends.
    - 1 (Exactly one)
    - 1..4 (A range from one to four)
    - * (Zero or many)
    - 1..* (At least one)
- **Roles:** Sometimes a role label (like "uses" or "drives") is written on the line to explain *how* they relate (e.g., A Rider *rides* a Cab).

B. Aggregation vs. Composition ("Has-A" Relationships)



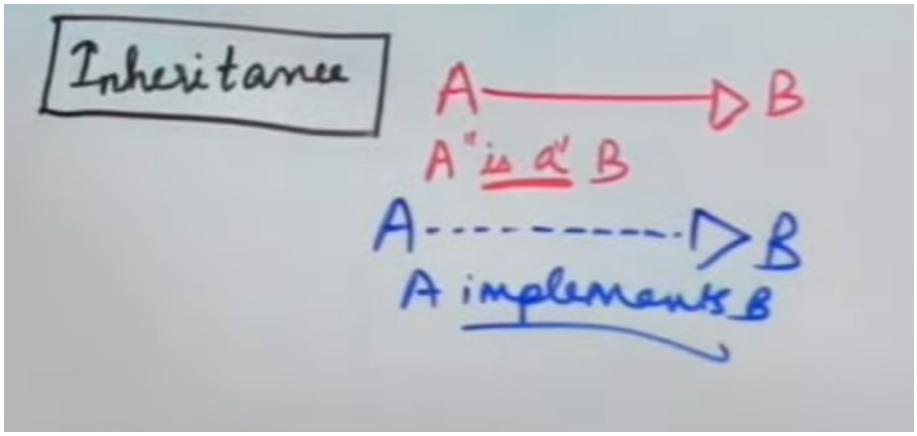


Both involve a parent-child relationship where one class "has an instance" of another class. They are denoted by diamond-headed arrows.

Feature	Aggregation	Composition
Symbol	Hollow/Empty Diamond	Filled/Solid Diamond
Lifecycle Dependency	Child **can** exist independently of the Parent.	Child **cannot** exist without the Parent.
Example	Desktop and Monitor. If the Desktop is destroyed, the Monitor can still exist and be	Body and Eye. The Eye is fundamentally a part of the Body; if the body ceases, the

Feature	Aggregation	Composition
	attached to another system.	eye does too (in an OOD context). Similarly, a Vehicle and its RC/Identifier.
Instance Sharing	The child instance can belong to multiple parent classes.	The entire existence of the child is managed by the specific instance of the parent class.

C. Inheritance ("Is-A" Relationship)



- Denoted by a **solid line with a hollow/empty triangle pointing to the Parent**.
- Represents a parent-child inheritance where the child "is a" type of the parent.
- **Example:** Person and Animal both inherit from LivingBeing.

D. Interfaces

- **Implementation:** When a class implements an interface, it is denoted by a **dotted line with a hollow/empty triangle** pointing to the interface.
- **Usage:** If a class simply *uses* an interface (but doesn't implement it), it is denoted by a **dotted line with an open arrow**.

4. Drawing Tools for Interviews

In machine coding rounds or LLD interviews, quickly sketching out these conceptual diagrams (even on a whiteboard or using basic text approximations) is valuable. For proper documentation, free online tools like Lucidchart or draw.io are highly recommended.